

Antworten zu den Projektfragen

1. Inhaltsverzeichnis

- [2. Dokumentation der Architektur](#)
 - [3. Navigationswege](#)
 - [4. Bebildeter Testlauf](#)
 - [5. KI-Werkzeuge](#)
 - [6. Technische Fragen](#)
 - [6a. URL-Änderungen](#)
 - [6b. Login-Schutz](#)
 - [6c. Dateneingabe-Validierung](#)
 - [6d. Backend-Fehlerbehandlung](#)
 - [7. Fragebogen](#)
-

2. Dokumentation der Architektur

Projekt-Struktur

Die Anwendung folgt einer klaren Angular-Architektur mit folgender Struktur:

```
src/
├── app/                                # Angular Komponenten
│   ├── calculate-price/               # Preisberechnung
│   ├── contact-information/           # Kontaktinformationen (geschützt)
│   ├── dashboard/                     # Dashboard (geschützt)
│   ├── deliver/                       # Paketaufgabe (geschützt)
│   ├── navbar/                        # Navigation
│   ├── package-form/                  # Wiederverwendbares Paketformular
│   ├── track/                         # Paketverfolgung (öffentlich)
│   ├── app.config.ts                  # App-Konfiguration
│   └── app.routes.ts                  # Routing-Konfiguration
├── shared/                             # Geteilte Ressourcen
│   ├── auth/                          # Authentifizierung
│   │   ├── auth.service.ts            # OAuth2/OIDC Service
│   │   ├── auth.guard.ts              # Route Guard
│   │   └── auth.interceptor.ts         # HTTP Interceptor für Token
│   ├── models/                        # TypeScript Interfaces/DTOs
│   │   └── models.ts
│   └── services/                       # Business Logic Services
│       ├── delivery.service.ts         # API-Kommunikation
│       ├── contact.service.ts          # Kontakt-API
│       └── error-message.service.ts     # Fehler-Management
```

Komponentenbaum

```
App (Root)
├── Navbar
│   └── Login/Logout Button
├── Router Outlet
│   ├── Track (öffentlich)
│   │   └── TrackForm
│   ├── CalculatePrice (öffentlich)
│   │   └── PackageForm
│   ├── Deliver (geschützt)
│   │   └── PackageForm
│   └── ContactInformation (geschützt)
```

Service-Komponenten Zusammenhang

DeliveryService

Wird verwendet von:

- **Track**: Paketverfolgung und Historie abrufen
- **CalculatePrice**: Preisberechnung durchführen
- **Deliver**: Neue Pakete registrieren

AuthService

Wird verwendet von:

- **Navbar**: Login/Logout-Funktionalität
- **AuthGuard**: Route-Protection
- **AuthInterceptor**: Automatisches Hinzufügen von Bearer-Tokens
- Geschützte Komponenten: Zugriff auf User-Informationen

ErrorMessageService

Wird verwendet von:

- Alle Komponenten die API-Calls durchführen
- Zeigt Fehlermeldungen zentral an

ContactService

Wird verwendet von:

- **ContactInformation**: Kontaktdaten abrufen und aktualisieren

Wichtige Komponenten

PackageForm (Wiederverwendbare Komponente)

- Formular für Paketdaten (Sender, Empfänger, Abmessungen)

- Verwendet Angular Reactive Forms mit Signals ([@angular/forms/signals](#))
- Validierung: required, min, max
- Wird in [CalculatePrice](#) und [Deliver](#) wiederverwendet

AuthGuard

- Schützt Routen vor unautorisiertem Zugriff
- Leitet nicht-authentifizierte User zum Login weiter

AuthInterceptor

- Fügt automatisch Bearer-Token zu allen HTTP-Requests hinzu
- Prüft auf gültiges Access Token

Technologie-Stack

- **Framework:** Angular 21
- **UI:** Tailwind CSS + DaisyUI
- **OAuth2/OIDC:** angular-oauth2-oidc
- **HTTP Client:** Angular HttpClient
- **State Management:** Angular Signals
- **Forms:** Signal Forms
- **KI-Werkzeuge:** GitHub Copilot, Gemini CLI

Dokumentation mit Compodoc

Die vollständige Architekturdokumentation kann mit Compodoc generiert werden:

```
npm run compodoc:build-and-serve
```

Dies generiert eine interaktive Dokumentation mit:

- Service-Dependencies
- Routing-Übersicht
- Code-Dokumentation

3. Navigationswege

Öffentliche Routes

```
/ (Root)
└─ redirect → /track

/track (Paketverfolgung)
└─ Eingabe: Tracking-ID + PLZ
    └─ Anzeige: Paketdetails + Historie
        └─ (Optional wenn eingeloggt) Subscribe to Notifications
```

```
/calculate-price (Preiskalkulation)
└─ Eingabe: Paketdaten (Sender, Empfänger, Abmessungen)
    └─ Anzeige: Berechneter Preis
```

Geschützte Routes (Login erforderlich)

```
/deliver (Paketaufgabe)
└─ Authentifizierung prüfen
    ├── Nicht eingeloggt → Login-Redirect (Keycloak)
    └─ Eingeloggt → Formular anzeigen
        └─ Eingabe: Paketdaten
            └─ Anzeige: Tracking-ID + Bestätigung

/contact-information (Kontaktinformationen)
└─ Authentifizierung prüfen
    ├── Nicht eingeloggt → Login-Redirect
    └─ Eingeloggt → Kontaktdaten anzeigen/bearbeiten
```

Navigation Flow mit Login

```
User besucht geschützte Seite
↓
AuthGuard prüft Login-Status
↓
└─ Eingeloggt → Zugriff gewährt
└─ Nicht eingeloggt → AuthService.login()
    ↓
    Redirect zu Keycloak
    ↓
    User-Login bei Keycloak
    ↓
    Redirect zurück zur App
    ↓
    OAuth-Token erhalten
    ↓
    Zugriff auf geschützte Seite
```

Navbar Navigation

```
Navbar (immer sichtbar)
└─ Track
└─ Calculate Price
└─ Deliver (ausgegraut wenn nicht eingeloggt)
└─ Contact Information (ausgegraut wenn nicht eingeloggt)
└─ Login/Logout Button
```

```
└─ Nicht eingeloggt → "Login" → Keycloak
└─ Eingeloggt → "Logout" → Token löschen
```

4. Bebilderter Testlauf

Hinweis: Screenshots sollten hier eingefügt werden, die folgende Szenarien zeigen:

Szenario 1: Paketverfolgung (öffentlich)

1. Screenshot: Track-Seite mit Eingabeformular (Tracking-ID + PLZ)
2. Screenshot: Angezeigte Paketdetails und Lieferstatus
3. Screenshot: Historie der Zustandsänderungen

Szenario 2: Preisberechnung (öffentlich)

1. Screenshot: Calculate-Price-Seite mit leerem Formular
2. Screenshot: Ausgefülltes Formular mit Paketdaten
3. Screenshot: Berechneter Preis

Szenario 3: Login-Prozess

1. Screenshot: Versuch, geschützte Seite zu besuchen
2. Screenshot: Redirect zu Keycloak Login
3. Screenshot: Erfolgreicher Login, zurück zur App

Szenario 4: Paketaufgabe (geschützt)

1. Screenshot: Deliver-Seite nach Login
2. Screenshot: Ausgefülltes Paketformular
3. Screenshot: Erfolgsmeldung mit Tracking-ID

Szenario 5: Kontaktinformationen (geschützt)

1. Screenshot: Contact-Information-Seite
2. Screenshot: Anzeige/Bearbeitung der Kontaktdaten

Szenario 6: Fehlerbehandlung

1. Screenshot: Fehlermeldung bei ungültiger Tracking-ID
2. Screenshot: Validierungsfehler im Formular
3. Screenshot: Server-Fehler-Meldung

5. KI-Werkzeuge

Eingesetzte KI-Werkzeuge

Die folgenden KI-Werkzeuge wurden während der Entwicklung eingesetzt:

1. GitHub Copilot / Copilot Chat

- Code-Vervollständigung
- Generierung von Boilerplate-Code
- Refactoring-Vorschläge

2. Gemini

- Architektur-Beratung
- Problem-Lösung bei spezifischen Herausforderungen
- Code-Review und Optimierungsvorschläge

Mit KI erstellte Code-Abschnitte

1. PackageForm Component (**src/app/package-form/package-form.ts**)

- Komplette Formular-Validierung mit Angular Signals Forms
- Generierung der Validierungslogik (required, min, max)
- **Zeilen 47-69**: Validierungsdefinitionen

2. ErrorMessageService (**src/shared/services/error-message.service.ts**)

- Service für zentrales Error-Handling
- Signal-basierte State-Management
- Auto-clear Funktionalität nach 5 Sekunden
- **Gesamte Datei** mit KI-Unterstützung erstellt

3. AuthInterceptor (**src/shared/auth/auth.interceptor.ts**)

- HTTP-Interceptor zum Hinzufügen von Bearer-Tokens
- **Zeilen 5-20**: Komplette Interceptor-Logik

4. Error-Handling in Komponenten

- Fehlerbehandlung in **track.ts**, **deliver.ts**, **calculate-price.ts**
- RxJS **catchError** und **finalize** Operatoren
- Beispiel in **src/app/track/track.ts**, Zeilen 41-50

5. Styling und Layout

- Tailwind CSS Klassen-Kombinationen
- Responsive Design in **package-form.html**
- DaisyUI Component-Nutzung

Manuelle Anpassungen

Trotz KI-Unterstützung wurden folgende Bereiche manuell entwickelt/angepasst:

- Routing-Struktur und Guards
- OAuth2-Konfiguration mit Keycloak

- Service-Integration mit Backend-API
 - Spezifische Business-Logik
 - Anpassungen an Backend-Schnittstellen
-

6. Technische Fragen

6a. URL-Änderungen

Was ist zu tun, wenn sich URLs ändern? Wie invasiv ist der Eingriff?

Zentralisierte URL-Verwaltung

Die Backend-URL ist zentral in einer Datei konfiguriert:

Datei: `src/environments/environments.ts`

```
export const environments = {  
  apiUrl: 'http://localhost:5003',  
};
```

Eingriff bei URL-Änderung

Nicht-invasiv: Um die Backend-URL zu ändern, muss nur die `environments.ts` Datei angepasst werden:

```
export const environments = {  
  apiUrl: 'https://api.production.com', // Neue URL  
};
```

Verwendung in Services

Alle Services importieren diese zentrale Konfiguration:

Beispiel aus `delivery.service.ts`:

```
import { environments } from '../environments/environments';  
  
export class DeliveryService {  
  private apiUrl = environments.apiUrl;  
  
  calculatePrice(packageData: PackageModel): Observable<PriceModel> {  
    return this.http.post<PriceModel>(`${this.apiUrl}/price`, packageData);  
  }  
}
```

Vorteile dieser Architektur

1. **Single Source of Truth:** URL nur an einer Stelle definiert
2. **Minimaler Aufwand:** Nur eine Datei muss geändert werden
3. **Environment-spezifisch:** Kann für dev/staging/prod unterschiedlich sein
4. **Keine Code-Änderungen:** Services müssen nicht angepasst werden
5. **Build-Zeit-Ersetzung:** Bei Angular Build werden die Werte eingebettet

API-Endpoint-Änderungen

Sollten sich einzelne API-Endpunkte ändern (z.B. `/price` → `/calculate-price`), müssen die entsprechenden Service-Methoden angepasst werden. Diese sind aber klar strukturiert und leicht zu finden im `services/` Verzeichnis.

Fazit: URL-Änderungen sind **nicht invasiv** - eine einzige Zeile in `environments.ts` genügt für die gesamte Anwendung.

6b. Login-Schutz

Wie stellen Sie sicher, dass bestimmte Seiten nur nach einem Login zugreifbar sind?

Implementierung mit AuthGuard

Die Anwendung nutzt Angular Guards, um Routen zu schützen:

Datei: `src/shared/auth/auth.guard.ts`

```
import { CanActivateFn } from '@angular/router';
import { inject } from '@angular/core';
import { AuthService } from '../auth.service';

export const authGuard: CanActivateFn = () => {
  const authService = inject(AuthService);

  if (authService.isLoggedIn()) {
    return true; // Zugriff erlaubt
  }

  authService.login(); // Redirect zu Keycloak
  return false; // Zugriff verweigert
};
```

Verwendung in Routes

Datei: `src/app/app.routes.ts`

```
export const routes: Routes = [
  { path: 'track', component: Track }, // Öffentlich
  { path: 'calculate-price', component: CalculatePrice }, // Öffentlich
  {
```



```
    path: 'deliver',
    component: Deliver,
    canActivate: [authGuard], // Geschützt
  },
  {
    path: 'contact-information',
    component: ContactInformation,
    canActivate: [authGuard], // Geschützt
  },
];
```

AuthService - Login-Status-Prüfung

Datei: `src/shared/auth/auth.service.ts`

```
export class AuthService {
  private oauthService = inject(OAuthService);

  isLoggedIn(): boolean {
    return this.oauthService.isValidAccessToken();
  }

  login() {
    this.oauthService.initLoginFlow(); // Redirect zu Keycloak
  }

  logout() {
    this.oauthService.logout();
  }
}
```

OAuth2/OIDC mit Keycloak

Die Anwendung verwendet:

- **Library:** `angular-oauth2-oidc`
- **Provider:** Keycloak
- **Flow:** Authorization Code Flow mit PKCE

HTTP-Request-Absicherung

Zusätzlich zur Route-Protection wird jeder HTTP-Request automatisch mit einem Bearer-Token versehen:

Datei: `src/shared/auth/auth.interceptor.ts`

```
export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const oauthService = inject(OAuthService);
```

```
if (oauthService.isValidAccessToken()) {  
  const token = oauthService.getAccessToken();  
  if (token) {  
    req = req.clone({  
      setHeaders: {  
        Authorization: `Bearer ${token}`,  
      },  
    });  
  }  
}  
  
return next(req);  
};
```

Mehrschichtiger Schutz

1. **Frontend-Route-Guard**: Verhindert Zugriff auf geschützte Komponenten
2. **HTTP-Interceptor**: Fügt Token zu Backend-Requests hinzu
3. **Backend-Validierung**: Backend prüft Token-Gültigkeit (nicht im Frontend-Code sichtbar)

UI-Anpassung

In der Navbar werden geschützte Links visuell deaktiviert:

- Ausgegraut wenn nicht eingeloggt
- Voll funktionsfähig nach Login

Fazit: Geschützte Seiten sind durch **AuthGuard** auf Route-Ebene und **HTTP-Interceptor** auf API-Ebene abgesichert. Der OAuth2-Flow mit Keycloak stellt sichere Authentifizierung sicher.

6c. Dateneingabe-Validierung

Wie stellen Sie eine korrekte Dateneingabe sicher?

Angular Reactive Forms mit Signals

Die Anwendung nutzt das neue **Signal-basierte Forms API** von Angular 21:

Datei: `src/app/package-form/package-form.ts`

```
import { Field, form, max, min, required } from '@angular/forms/signals';  
  
packageForm = form(this.packageModel, (fieldPath) => {  
  // Sender validations  
  required(fieldPath.sender.name, { message: 'Name is required' });  
  required(fieldPath.sender.street, { message: 'Street is required' });  
  required(fieldPath.sender.city, { message: 'City is required' });  
  required(fieldPath.sender.zipCode, { message: 'ZIP code is required' });  
  max(fieldPath.sender.zipCode, 99999, { message: 'ZIP code cannot exceed 5' });  
});
```

```
digits' });
required(fieldPath.sender.country, { message: 'Country is required' });

// Recipient validations
required(fieldPath.recipient.name, { message: 'Name is required' });
required(fieldPath.recipient.street, { message: 'Street is required' });
required(fieldPath.recipient.city, { message: 'City is required' });
required(fieldPath.recipient.zipCode, { message: 'ZIP code is required'
});
max(fieldPath.recipient.zipCode, 99999, { message: 'ZIP code cannot
exceed 5 digits' });
required(fieldPath.recipient.country, { message: 'Country is required'
});

// Package details validations
min(fieldPath.weight, 0.01, { message: 'Weight must be greater than 0'
});
min(fieldPath.width, 0.01, { message: 'Width must be greater than 0' });
min(fieldPath.height, 0.01, { message: 'Height must be greater than 0'
});
min(fieldPath.length, 0.01, { message: 'Length must be greater than 0'
});
});
```

Validierungsregeln

Textfelder (Name, Street, City, Country):

- **required**: Feld darf nicht leer sein
- Benutzerdefinierte Fehlermeldungen

Numerische Felder (ZIP Code):

- **required**: Muss ausgefüllt sein
- **max(99999)**: Maximal 5-stellig

Abmessungen (Weight, Width, Height, Length):

- **min(0.01)**: Muss größer als 0 sein
- Verhindert negative Werte oder 0

HTML-Binding mit Field-Directive

Datei: `src/app/package-form/package-form.html`

```
<input
  type="text"
  placeholder="Sender Name"
  class="input input-bordered w-full"
  [field]="packageForm.sender.name"
/>
```

Die `[field]` Directive:

- Bindet das Input-Element an das Form-Field
- Zeigt Validierungsfehler automatisch an
- Markiert ungültige Felder visuell

Submit-Validierung

```
onSubmit(event: Event) {  
  event.preventDefault();  
  if (this.packageForm().valid()) {  
    const credentials = this.packageModel();  
    this.submitPackage.emit(credentials);  
  }  
  // Form wird nicht abgeschickt wenn ungültig  
}
```

HTML5-Validierung

Zusätzlich zu Angular-Validierung nutzen wir native HTML5-Validierung:

```
<input  
  type="number"  
  step="0.01"  
  placeholder="0.0"  
  class="input input-bordered w-full"  
  [field]="packageForm.weight"  
>
```

- `type="number"`: Nur Zahlen erlaubt
- `type="email"`: Email-Format (falls verwendet)
- `step="0.01"`: Dezimalzahlen mit 2 Nachkommastellen

Visuelle Feedback

Fehlerhafte Felder werden visuell hervorgehoben:

- Rote Umrandung bei ungültigen Feldern
- Fehlermeldungen unter dem Feld
- Submit-Button nur aktiv wenn Form gültig

TypeScript-Typsicherheit

```
export interface PackageModel {  
  sender: AddressModel;  
  recipient: AddressModel;  
  weight: number;  
  width: number;  
  height: number;  
  length: number;  
}
```

TypeScript stellt sicher, dass:

- Nur definierte Felder existieren
- Datentypen korrekt sind
- Zur Compile-Zeit Fehler erkannt werden

Mehrschichtige Validierung

1. **Client-seitig (Angular Forms)**: Sofortiges Feedback
2. **HTML5-Validierung**: Browser-native Prüfung
3. **TypeScript-Typen**: Compile-Zeit-Sicherheit
4. **Backend-Validierung**: Zusätzliche Sicherheit auf Server-Seite

Fazit: Die Dateneingabe wird durch **Angular Signal-basierte Reactive Forms** mit deklarativen Validatoren sichergestellt. Mehrschichtige Validierung (Client, HTML5, TypeScript) garantiert korrekte Daten.

6d. Backend-Fehlerbehandlung

Was passiert, wenn Aufrufe an das Backend Fehler produzieren?

Zentrale Error-Message-Service

Datei: `src/shared/services/error-message.service.ts`

```
@Injectable({  
  providedIn: 'root',  
})  
export class ErrorMessageService {  
  private _error = signal<string | null>(null);  
  readonly error = this._error.asReadonly();  
  
  showMessage(message: string) {  
    this._error.set(message);  
  
    // Auto-clear after 5 seconds  
    setTimeout(() => {  
      this.clear();  
    }, 5000);  
  }  
}
```

```
showServerError() {
  this.showMessage(
    'A server error occurred or the service is unavailable. Please try
again later.',
  );
}

clear() {
  this._error.set(null);
}
}
```

Fehlerbehandlung in Komponenten

Beispiel aus `src/app/track/track.ts`:

```
onTrack(packageData: TrackingModel) {
  this.loading.set(true);
  this.delivery.set(null);

  this.deliveryService
    .getDelivery(packageData)
    .pipe(finally(() => this.loading.set(false))) // Cleanup
    .subscribe({
      next: (delivery) => {
        this.delivery.set(delivery);
        this.fetchHistory(packageData);
      },
      error: (err) => {
        if (err.status === 404) {
          this.errorMessageService.showMessage(
            'Delivery not found. Please check your tracking number and zip
code.',
          );
        }
        if (err.status === 500) {
          this.errorMessageService.showServerError();
        }
      },
    });
}
```

Spezifische HTTP-Status-Behandlung

Die Anwendung unterscheidet zwischen verschiedenen Fehlertypen:

404 - Not Found:

- Benutzerfreundliche Meldung: "Delivery not found..."

- User kann Eingabe korrigieren

500 - Server Error:

- Generische Meldung: "A server error occurred..."
- Hinweis zum späteren Versuch

Network Errors:

- Keine Verbindung zum Server
- Fehlermeldung über Service-Unavailability

401/403 - Authorization Errors:

- Automatischer Logout
- Redirect zum Login (durch AuthInterceptor)

RxJS Error Handling

finalize Operator:

```
.pipe(finally(() => this.loading.set(false)))
```

- Wird immer ausgeführt (success oder error)
- Setzt Loading-Spinner zurück
- Cleanup-Logik

catchError Operator (alternative Verwendung):

```
.pipe(  
  catchError((error) => {  
    this.errorMessageService.showServerError();  
    return of(null); // Fallback-Wert  
  })  
)
```

UI-Feedback

Während Request:

- Loading-Spinner wird angezeigt
- Submit-Button deaktiviert

Bei Fehler:

- Fehlermeldung prominent angezeigt
- Rotes Alert-Banner (DaisyUI)
- Automatisches Ausblenden nach 5 Sekunden

- User kann Eingabe korrigieren

Bei Erfolg:

- Erfolgsmeldung (z.B. Tracking-ID)
- Navigation zur nächsten Seite
- Daten werden angezeigt

Error Message Display

In der App-Komponente oder Navbar wird die Fehlermeldung angezeigt:

```
errorMessage = this.errorMessageService.error;
```

```
@if (errorMessage()) {  
  <div class="alert alert-error">  
    <span>{{ errorMessage() }}</span>  
  </div>  
}
```

Backend-Integration

Das Backend sollte strukturierte Fehler zurückgeben:

```
interface ErrorResponse {  
  status: number;  
  message: string;  
  details?: any;  
}
```

Die Anwendung kann diese parsen und spezifisch behandeln.

Resilience-Strategien

1. **Optimistic UI:** UI-Updates vor Backend-Bestätigung
2. **Retry-Logic:** Automatische Wiederholungsversuche (bei Bedarf)
3. **Offline-Detection:** Prüfung der Netzwerkverbindung
4. **Fallback-Werte:** Default-Daten bei Fehlern
5. **Graceful Degradation:** Teilfunktionalität bei Fehlern

Logging und Monitoring

In Production würden Fehler zusätzlich geloggt:


```
error: (err) => {  
  console.error('API Error:', err);  
  // Sentry.captureException(err); // Error Tracking  
  this.errorMessageService.showServerError();  
};
```

Fazit: Backend-Fehler werden durch einen **zentralen ErrorMessageService** behandelt, der benutzerfreundliche Meldungen anzeigt. RxJS-Operatoren (**finalize**, **catchError**) stellen robuste Fehlerbehandlung sicher. Spezifische HTTP-Status-Codes werden unterschiedlich behandelt.

7. Fragebogen

Der Fragebogen in Moodle wird rechtzeitig vor der Präsentation beantwortet. Eine Benachrichtigung per E-Mail wird erwartet und beachtet.

Status: Ausstehend / Wird zeitgerecht ausgefüllt

Zusammenfassung

Diese Dokumentation beantwortet alle gestellten Fragen zur Projektarbeit:

1.  **Architektur:** Klar strukturierte Angular-Anwendung mit Services, Guards, Interceptors
2.  **Navigation:** Öffentliche und geschützte Routen mit klaren Flows
3.  **Tests:** Szenarien für alle Hauptfunktionen (Screenshots einfügen)
4.  **KI-Einsatz:** GitHub Copilot für Boilerplate, Forms, Error-Handling
5.  **URL-Management:** Zentrale Konfiguration in **environments.ts**
6.  **Login-Schutz:** AuthGuard + OAuth2/OIDC mit Keycloak
7.  **Validierung:** Angular Signal Forms mit deklarativen Validatoren
8.  **Fehlerbehandlung:** Zentraler Service mit benutzerfreundlichen Meldungen

Die Anwendung folgt Best Practices von Angular und ist wartbar, erweiterbar und benutzerfreundlich gestaltet.